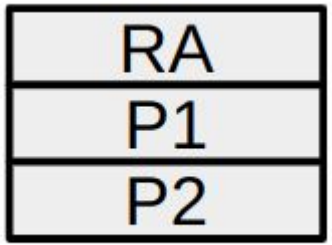
ED1 02: Structs e Ponteiros.

Fabio Irigon Pereira

Structs em C

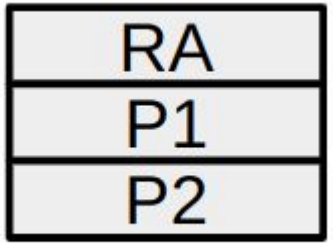
- Structs permitem agrupar diferentes tipos de dados em uma única unidade.
- São úteis para representar objetos mais complexos, como nós de listas, árvores e grafos.



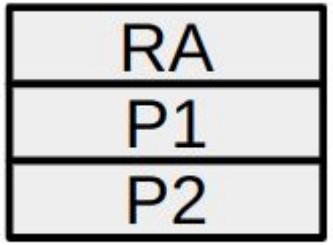
Definição de Structs em C

```c
#include <stdio.h>

struct aluno {
    int ra;
    float p1;
    float p2;
};
```



Uso de Structs em C




```c
#include <stdio.h>

struct aluno {
    int ra;
    float p1;
    float p2;
};

int main() {
    struct aluno a1 = {123456, 6.0, 7.5};
    printf("RA: %d, p1: %.2f, p2: %.2f\n" , a1.ra, a1.p1, a1.p2);
    return 0;
}
```

Exercício

Defina o elemento pessoa contendo nome e idade.

Crie um array structs pessoa, pré-preenchida.

Implemente uma função para contar quantas pessoas tem idade maior que 17 anos.

Structs e typedef

No exemplo temos que repetir o termo “struct” ao instanciar uma unidade:

```
struct aluno {
    …
};

int main() {
    struct aluno a1 = {123456, 6.0, 7.5};
    …
}
```

Structs e typedef

No exemplo temos que repetir o termo “struct” ao instanciar uma unidade:

```
struct aluno {
    …
};

int main() {
    struct aluno a1 = {123456, 6.0, 7.5};
    …
}
```

Structs e typedef

O typedef elimina essa necessidade

```
typedef struct {
    …
} aluno;

int main() {
    aluno a1 = {123456, 6.0, 7.5};
    …
}
```

Structs e typedef

Mas quando precisamos de um ponteiro para a própria struct dentro da struct:

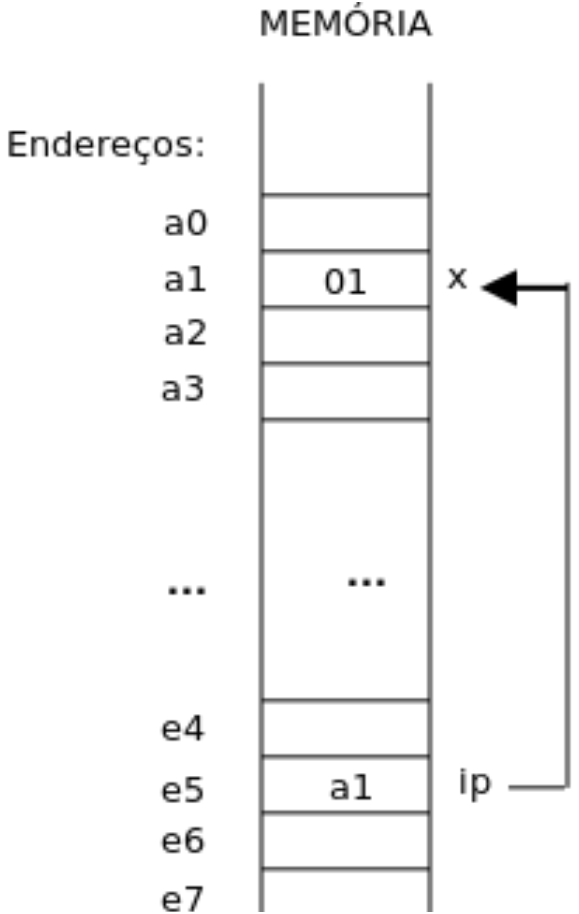
```
typedef struct aluno{
    …
    struct aluno *p;
} aluno;

int main() {
    aluno a1 = {123456, 6.0, 7.5};
    …
}
```

Ponteiros em C

O que são ponteiros?

- Ponteiros armazenam endereços de memória.
- Permitem manipular dados diretamente na memória, sendo fundamentais para alocação dinâmica e manipulação de **listas, árvores e grafos**.



Ponteiros em C

```c
int main() {
    int a = 10;
    int *ptr = &a;   // Ponteiro apontando para 'a'

    printf("Valor de a: %d\n", a);
    printf("Endereço de a: %p\n", &a);
    printf("Valor apontado por ptr: %d\n", *ptr);

    return 0;
}
```

Atenção para os operadores **‘&’** (endereço da variável) e **‘*’** (valor apontado pelo ponteiro.

Structs e Ponteiros juntos



Ponteiros e structs

No exemplo a função recebe um ponteiro para uma struct, e na main é passado o endereço da variável:

```c
#include <stdio.h>

void mostrarPessoa(Pessoa *p) {
    printf("Nome: %s, Idade: %d\n", p->nome, p->idade);
}

int main() {
    Pessoa p1 = {"João", 22};
    mostrarPessoa(&p1);   // Passando a struct por referência (O(1) vs O(n))
    return 0;
}
```

Ponteiros e structs

```c
#include <stdio.h>

void mostrarPessoa(Pessoa *p) {
    printf("Nome: %s, Idade: %d\n", p->nome, p->idade);
}

int main() {
    Pessoa p1 = {"João", 22};
    mostrarPessoa(&p1);   // Passando a struct por referência
    return 0;
}
```

Atenção para o operador ‘**->**’ (valor do campo da estrutura apontada pelo ponteiro).

Alocação dinâmica (malloc)

No exemplo abaixo uma struct é alocada dinamicamente:

```
Pessoa *a = (Pessoa *)malloc( sizeof(Pessoa));

if (a == NULL) { // testa se conseguiu alocar
    printf("Erro ao alocar memória\n");
    return 1;
}

// preenche a struct
a->idade = 10;
snprintf(a->nome, sizeof(a->nome), "Maria");

free(a);  // Libera memória alocada
```

Alocação dinâmica (malloc)

No exemplo abaixo uma struct é alocada dinamicamente:

```
Pessoa *a = (Pessoa *)malloc( sizeof(Pessoa));

if (a == NULL) { // testa se conseguiu alocar
    printf("Erro ao alocar memória\n");
    return 1;
}

// preenche a struct
a->idade = 10;
snprintf(a->nome, sizeof(a->nome), "Maria");

free(a);  // Libera memória alocada
```

"O `malloc` é como alugar um quarto de hotel; se você sair e não devolver a chave (`free`), o quarto fica ocupado para sempre, mas ninguém o está usando"

Atenção

O que faz este código?

```c
typedef struct {
    int id;
    char nome[30];
} Aluno;

int main() {
    Aluno *a;
    printf("%s", a->nome);
    return 1;
}
```

Atenção

O que faz este código?

```
typedef struct {
    int id;
    char nome[30];
} Aluno;

int main() {
    Aluno *a;
    printf("%s", a->nome);
    return 1;
}
```

Quando usamos ponteiros, é fácil cometer erros que geram falha de segmentação, que muitas vezes são difíceis de encontrar.



Exercício:

Crie uma struct chamada `Produto` com os campos:

- `char nome[50]`
- `float preco`
- `int estoque`

No `main()`, declare três produtos, preencha seus dados manualmente e imprima as informações na tela.

Faça uma função `void atualizarEstoque(Produto *p, int quantidade)` para atualizar o estoque usando ponteiros.

Exercício 2:

Crie uma struct chamada `Retangulo` com os campos:

- `float largura`
- `float altura`

Implemente as seguintes funções:

- `float calcularArea(Retangulo *r)`: retorna a área do retângulo.
- `float calcularPerimetro(Retangulo *r)`: retorna o perímetro.

No `main()`, leia os valores do usuário, chame as funções e exiba os resultados.

Structs encadeadas e Ponteiros para ponteiros

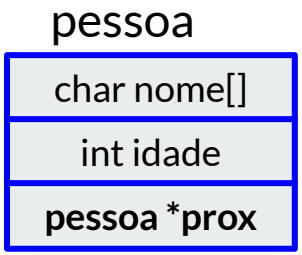




Structs com ponteiros

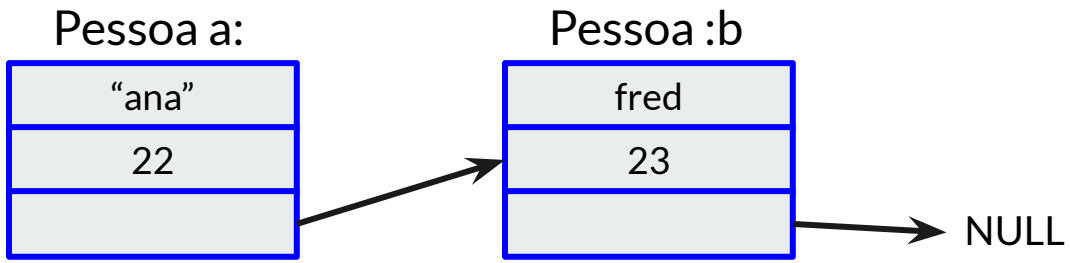
Note que a struct abaixo contém um ponteiro para a própria struct.

```
typedef struct Pessoa{
    char nome[50];
    int idade;
    struct Pessoa *prox;
}Pessoa;
```



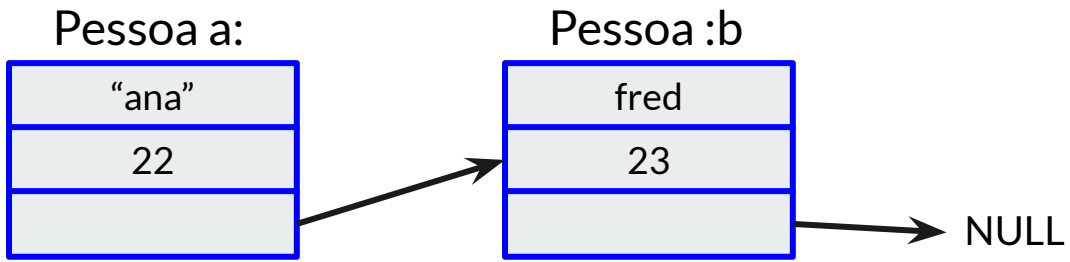
Encadear estruturas

Isso permite construir uma cadeia de structs
(análogo a um trem de vagões)



Encadear estruturas

Isso permite construir uma cadeia de structs
(análogo a um trem de vagões)




```
O que seria impresso com este comandos?
printf("%s", a->nome);
printf("%s", a->prox->nome);
printf("%s", b->prox->nome);
```

Teste de mesa

Desenhe três caixas representando a memória e rastreie o valor de cada variável e para onde cada ponteiro aponta após a execução de cada linha.

```
typedef struct No{
    int id;
    No *prox;
} No;

No a = {10, NULL};
No b = {20,NULL};
No *p = &a;

p->prox = &b; // Linha 1
p = p->prox; // Linha 2
p->id = 30; // Linha 3
```

Ponteiro para ponteiro para struct

Se um ponteiro é passado por cópia para uma função, seu conteúdo não será modificado.

Nestes casos utilizamos dupla indireção.

```
Pessoa aluno;
Pessoa *p = &aluno;
Pessoa **pp = &p;
```

```
printf("%s", aluno.nome);
printf("%s", p->nome);
printf("%s", (*pp)->nome);
```